

Prova II - Teoria

Entrega 8 nov em 10:40**Pontos** 10**Perguntas** 9**Disponível** 8 nov em 8:40 - 8 nov em 10:40 aproximadamente 2 horas**Limite de tempo** 120 Minutos

Instruções

Nossa segunda prova de AEDs II tem duas partes: teórica e prática. Cada uma vale 10 pontos. A prova teórica será realizada no Canvas e a prática, no Verde.

A prova teórica terá 9 questões sendo 6 questões fechadas e 3 abertas. Cada fechada vale 0.5 pontos; a primeira aberta, 3 pontos e as demais abertas, 2 pontos cada. Após terminar uma questão, o aluno **NÃO** terá a opção de retornar à mesma. No caso das questões abertas, o aluno deverá enviar um arquivo contendo sua resposta. Essa resposta pode ser uma imagem (foto de boa resolução) ou um código fonte. No horário da aula, você pode acessar a Prova II através do menu "Testes / Prova II".

Este teste foi travado 8 nov em 10:40.

Histórico de tentativas

	Tentativa	Tempo	Pontuação
MAIS RECENTE	Tentativa 1	117 minutos	9,5 de 10

❗ As respostas corretas não estão mais disponíveis.

Pontuação deste teste: **9,5** de 10

Enviado 8 nov em 10:37

Esta tentativa levou 117 minutos.

Pergunta 1

0,5 / 0,5 pts

Os conceitos de endereço e ponteiro são trabalhados de forma explícita na linguagem C. Em relação aos ponteiros, avalie as afirmações a seguir.

I. Após a sequência de instruções abaixo, *p é igual a 5.

```
int *p;  
int x = 5;  
p = &x;
```

II. A saída na tela abaixo quando o usuário digita 15 como entrada de dados será 10 15

iii. Referente a alocação dinâmica de memória em C, as funções calloc e realloc são usadas para liberar arrays.

É correto o que se afirma em

☐ II, apenas.

☐ I, II e III.

☐ II e III, apenas.

☒ I, apenas.

☐ I e III, apenas.

Execute os dois códigos. No caso da terceira afirmação, em C, quem libera espaço é a função free.

Pergunta 2

0,5 / 0,5 pts

Uma lista encadeada é uma representação de uma sequência de registros na memória *Random Access Memory* (RAM) do computador. Cada elemento da sequência é armazenado em uma célula da lista sendo o primeiro elemento na primeira célula, o segundo na segunda, e assim por diante. Considere o código abaixo contendo um registro do tipo `NoLista` e uma função para inserir no início da lista (UFS'14 - adaptada).

```
struct NoLista {  
    int elemento;  
    struct NoLista *proximo;  
};  
  
struct NoLista *inserirInicio(struct NoLista *inicio, int num, int *erro){  
    struct NoLista *novo;  
    *erro = 0;  
    novo = (struct NoLista*) malloc(sizeof(struct NoLista));  
    if (novo == NULL){  
        *erro = 1;  
        return inicio;  
    } else {
```

```
    novo->elemento=num;
    _____ /* (1) */
    return novo;
}
}
```

Para que a função, que insere um novo elemento no início da lista e retorne o início da lista, funcione corretamente, a linha em branco, marcada com o comentário (1), deve ser preenchida com

- ☐ novo = inicio;
- ☐ inicio = novo;
- ☐ novo->proximo = inicio->proximo;
- ☐ inicio->proximo = novo;
- ☒ novo->proximo = inicio;

O comando /* (1) */ será "*novo->proximo = inicio;*". Nesse caso, temos que o ponteiro novo aponta para a nova célula que contém o novo item e seu "próximo" aponta para o início atual. Quando a função for chamada, o novo valor de início deve ser atualizado com o endereço da célula recém criada que é retornado pela função em questão.

Pergunta 3

0,5 / 0,5 pts

Os conjuntos são tão fundamentais para a ciência da computação quanto o são para a matemática. Enquanto os conjuntos matemáticos são invariáveis, os conjuntos manipulados por algoritmos podem crescer, encolher ou sofrer outras mudanças ao longo do tempo. Chamamos tais conjuntos de conjuntos dinâmicos.

CORMEN T. H., et al., **Algoritmos: teoria e prática**, 3ed, Elsevier, 2012

A figura apresentada abaixo contém uma lista duplamente encadeada, um tipo de conjunto dinâmico.



O algoritmo abaixo é manipula os elementos de uma lista duplamente encadeada.

```
Celula *i = primeiro->prox;
Celula *j = ultimo;
Celula *k;
while (i != j && j->prox != i){
    int tmp = i->elemento;
    i->elemento = j->elemento;
    j->elemento = tmp;
    i = i->prox;
```

```
for (k = primeiro; k->prox != j; k = k->prox); j = k;  
}
```

Considerando a figura e o código acima e seus conhecimentos sobre listas duplamente encadeadas, avalie as afirmações a seguir:

I. A execução do algoritmo na lista faz com que o conteúdo da lista (da primeira para a última célula) seja: 0 5 4 3 2.

II. A condição $(i \neq j \ \&\& \ j \rightarrow \text{prox} \neq i)$ permite que o código funcione quando nossa lista tem tamanho par ou ímpar.

III. Na lista original, a execução do comando primeiro->prox->prox->prox->prox->ant->elemento exibe na tela o número 4.

É correto o que se afirma em

☒ I, II e III.

☐ I e III, apenas.

☐ I e II, apenas.

☐ II, apenas.

☐ III, apenas.

I CORRETA. O algoritmo em questão inverte a ordem dos elementos. A ordem do primeiro elemento é mantida dado que ele é o nó cabeça.

II. CORRETA - A condição $i \neq j$ aborta o laço quando a quantidade de elementos úteis é par. A $j \rightarrow \text{prox} \neq i$, quando essa quantidade é par.

III. CORRETA. A execução do comando primeiro $\rightarrow$ prox $\rightarrow$ prox $\rightarrow$ prox $\rightarrow$ prox $\rightarrow$ ant $\rightarrow$ elemento exibe na tela o número 4.

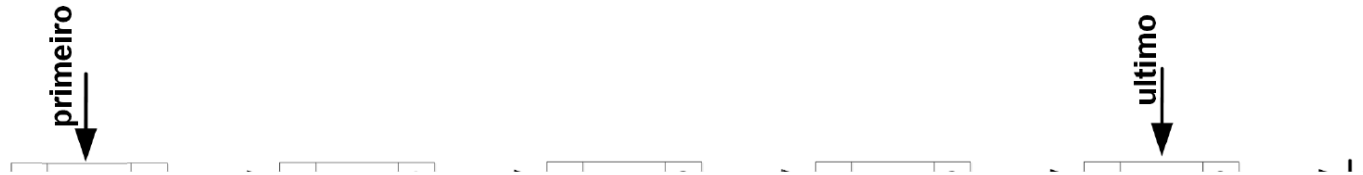
Pergunta 4

0,5 / 0,5 pts

Os conjuntos são tão fundamentais para a ciência da computação quanto o são para a matemática. Enquanto os conjuntos matemáticos são invariáveis, os conjuntos manipulados por algoritmos podem crescer, encolher ou sofrer outras mudanças ao longo do tempo. Chamamos tais conjuntos de conjuntos dinâmicos.

CORMEN T. H., et al., **Algoritmos: teoria e prática**, 3ed, Elsevier, 2012

A figura apresentada abaixo contém uma lista duplamente encadeada, um tipo de conjunto dinâmico.



O algoritmo abaixo é manipula os elementos de uma lista duplamente encadeada.

```

Celula *i = primeiro->prox;
Celula *j = ultimo;
Celula *k;
while (i != j && j->prox != i){
    int tmp = i->elemento;
    i->elemento = j->elemento;
    j->elemento = tmp;
    i = i->prox;
    for (k = primeiro; k->prox != j; k = k->prox); j = k;
}

```

Considerando a figura e o código acima e seus conhecimentos sobre listas duplamente encadeadas, avalie as afirmações a seguir:

I. A execução do algoritmo na lista faz com que o conteúdo da lista (da primeira para a última célula) seja: 0 7 5 3 1.

II. A condição $(i \neq j \ \&\& \ j \rightarrow \text{prox} \neq i)$ permite que o código funcione quando nossa lista tem tamanho par ou ímpar.

III. Na lista original, a execução do comando `primeiro->prox->prox->prox->prox->ant->elemento` exibe na tela o número 7.

É correto o que se afirma em

- ☐ II, apenas.
- ☐ I, II e III.
- ☐ III, apenas.
- ☐ I e III, apenas.
- ☒ I e II, apenas.

I CORRETA. O algoritmo em questão inverte a ordem dos elementos. A ordem do primeiro elemento é mantida dado que ele é o nó cabeça.

II. CORRETA - A condição $i \neq j$ aborta o laço quando a quantidade de elementos úteis é par. A $j \rightarrow prox \neq i$, quando essa quantidade é par.

III. ERRADA. A execução do comando primeiro $\rightarrow prox \rightarrow prox \rightarrow prox \rightarrow prox \rightarrow ant \rightarrow elemento$ exibe na tela o número 5.

Pergunta 5

0,5 / 0,5 pts

Uma lista encadeada é uma estrutura de dados flexível composta por células sendo que cada célula tem um elemento e aponta para a próxima célula da lista. A última célula aponta para *null*. A lista encadeada tem dois ponteiros: primeiro e último. Eles apontam para a primeira e última célula da lista, respectivamente. A lista é dita duplamente encadeada quando cada célula tem um ponteiro anterior que aponta para a célula anterior. O ponteiro anterior da primeira célula aponta para *null*. A respeito das listas duplamente encadeadas, avalie as asserções a seguir.

I. Na função abaixo em C para remover a última célula, o comando "*free(ultimo->prox)*" pode ser eliminado mantendo o funcionamento de nossa lista para outras operações. Contudo, isso pode gerar um vazamento de memória porque o espaço de memória relativo à célula "removida" só será liberado no final da execução do programa.

```
int removerFim() {  
    if (primeiro == ultimo) errx(1, "Erro!");  
  
    int elemento = ultimo->elemento;  
    ultimo = ultimo->ant;  
    ultimo->prox->ant = NULL;  
    free(ultimo->prox);  
    ultimo->prox = NULL;  
    return elemento;  
}
```

PORQUE

II. As linguagem C e C++ não possuem coleta automática de lixo como na linguagem JAVA. Essa coleta do JAVA é realiza pela Máquina Virtual Java que reivindica a memória ocupada por objetos que não são mais acessíveis.

A respeito dessas asserções, assinale a opção correta.

- ☒ As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- ☐ As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- ☐ As asserções I e II são proposições falsas.
- ☐ A asserção I é uma proposição verdadeira, e a asserção II é uma proposição falsa.
- ☐ A asserção I é uma proposição falsa, e a asserção II é uma proposição verdadeira.

As duas afirmações são verdadeiras e a segunda justifica a primeira. O comando *free(ultimo->prox)* serve para eliminar uma célula que não é mais referenciada pela nossa estrutura. Como as linguagens C/C++ não possuem coleta de lixo, o programador é o responsável por essa tarefa. A falta dessa tarefa mantém o funcionamento das demais operações da lista, contudo, isso pode gerar um vazamento de memória.

Pergunta 6

0,5 / 0,5 pts

Uma lista encadeada é uma estrutura de dados flexível composta por células sendo que cada célula tem um elemento e aponta para a próxima célula da lista. A última célula aponta para *null*. A lista encadeada tem dois ponteiros: primeiro e último. Eles apontam para a primeira e última célula da lista, respectivamente. A lista é dita duplamente encadeada quando cada célula tem um ponteiro anterior que aponta para a célula

anterior. O ponteiro anterior da primeira célula aponta para *null*. A respeito das listas duplamente encadeadas, avalie as asserções a seguir.

I. Na função abaixo em C para remover a última célula, o comando "*tmp = NULL*" pode ser eliminado mantendo o funcionamento de nossa lista para outras operações. Contudo, isso pode gerar um vazamento de memória porque o espaço de memória relativo à célula ``removida" só será liberado no final da execução do programa.

```
int removerInicio() {  
    if (primeiro == ultimo) errx(1, "Erro!");  
  
    CelulaDupla *tmp = primeiro;  
    primeiro = primeiro->prox;  
    int elemento = primeiro->elemento;  
    primeiro->ant = NULL;  
    tmp->prox = NULL  
    free(tmp);  
    tmp = NULL;  
    return elemento;  
}
```

PORQUE

II. As linguagem C e C++ não possuem coleta automática de lixo como na linguagem JAVA. Essa coleta do JAVA é realiza pela Máquina Virtual Java que reivindica a memória ocupada por objetos que não são mais acessíveis.

A respeito dessas asserções, assinale a opção correta.

- ☐ A asserção I é uma proposição verdadeira, e a asserção II é uma proposição falsa.

- ☐ As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
- ☐ As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- ☐ As asserções I e II são proposições falsas.
- ☒ A asserção I é uma proposição falsa, e a asserção II é uma proposição verdadeira.

A primeira afirmação é falsa. Realmente, o comando `tmp = NULL` pode ser eliminado mantendo o funcionamento de nossa lista para outras operações. Isso, porque a variável `tmp` é uma variável local cujo escopo de vida limita-se a função em questão. A remoção do comando citado não causa qualquer vazamento de memória.

A segunda afirmação é verdadeira dado que as linguagem C e C++ não possuem coleta automática de lixo e o Java possui.

Pergunta 7

2,5 / 3 pts

Suponha uma árvore binária (não necessariamente de pesquisa) de caracteres e faça um método eficiente que retorna true se o caminho entre a raiz e alguma folha tiver duas vogais consecutivas. Em seguida, mostre a complexidade do seu método.

↓ [P02Q07.java \(https://pucminas.instructure.com/files/4869148/download\)](https://pucminas.instructure.com/files/4869148/download)

```
boolean busca(){
    return busca(raiz);
}
boolean condicao(char c){
    boolean resp;
    c = (c >= 'a' && c <= 'z') ? (char)(c - 32) : (c);
    return (c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U');
}
boolean busca(No i){
    boolean resp = false;
    if(i != null){
        resp = condicao(i.elemento) && ((i.esq != null && condicao(i.esq)) || (i.dir != null &&
condicao(i.dir)));
        if(resp == false){
            resp = busca(i.esq);
            if(resp == false){
                resp = busca(i.dir);
            }
        }
    }
    return resp;
} //O método faz O(n) visitas onde n é o número de nós da árvore
```

Pergunta 8**2 / 2 pts**

Seja nossa classe **Pilha Flexível**, faça um método **R.E.C.U.R.S.I.V.O** que recebe três objetos do tipo Pilha e mostra na tela a soma do primeiro elemento das três pilhas, a do segundo, do terceiro e, assim, sucessivamente. Lembre-se que as pilhas podem ter tamanhos distintos.

Nesta questão você deve apresentar o código fonte da solução e explicá-lo via vídeo.

Sua Resposta:

```
class Celula {
    public int elemento; // Elemento inserido na celula.
    public Celula prox; // Aponta a celula prox.

    public Celula() {
        this(0);
    }

    public Celula(int elemento) {
        this.elemento = elemento;
        this.prox = null;
    }
}

class Pilha {
    private Celula topo;

    public Pilha() {
        topo = null;
    }

    public void inserir(int x) {
        Celula tmp = new Celula(x);
```

```
    tmp.prox = topo;
    topo = tmp;
    tmp = null;
}

public void mostrar() {
    System.out.print("[ ");
    for (Celula i = topo; i != null; i = i.prox) {
        System.out.print(i.elemento + " ");
    }
    System.out.println("] ");
}

public int getSoma(Pilha pilha1, Pilha pilha2) {
    return getSoma(topo, pilha1.topo, pilha2.topo);
}

private int getSoma(Celula i, Celula i1, Celula i2) {
    int resp = 0;
    if (i != null && i1 != null && i2 != null) {
        resp = i.elemento + i1.elemento + i2.elemento;
        getSoma(i.prox, i1.prox, i2.prox);
        System.out.print(resp + " ");
    }
    return resp;
}
}

public class P02Q08{
    public static void main(String[] args) throws Exception{
```



```
Pilha pilha1 = new Pilha();  
Pilha pilha2 = new Pilha();  
Pilha pilha3 = new Pilha();
```

```
pilha1.inserir(1);  
pilha1.inserir(2);  
pilha1.inserir(3);  
pilha1.inserir(4);  
pilha1.inserir(5);
```

```
pilha2.inserir(5);  
pilha2.inserir(4);  
pilha2.inserir(3);  
pilha2.inserir(2);  
pilha2.inserir(1);
```

```
pilha3.inserir(2);  
pilha3.inserir(1);  
pilha3.inserir(5);  
pilha3.inserir(3);  
pilha3.inserir(4);
```

```
System.out.print("Soma das 3 pilhas: [ ");  
pilha1.getSoma(pilha2, pilha3);  
System.out.print("]");
```

```
System.out.print("\nFila 1: ");  
pilha1.mostrar();  
System.out.print("Fila 2: ");  
pilha2.mostrar();
```

```
System.out.print("Fila 3: ");  
pilha3.mostrar();  
}  
}
```

Nesta questão o aluno deve *desempilhar* e exibir um elemento de cada pilha, intercalando. Assim, caso uma das pilha fique vazia primeiro, a exibição deve continuar até que a outra estrutura fique vazia. Atenção as consistências que devem ser feitas para cada estrutura, verificando se a mesma esta ou não vazia.

Pergunta 9

2 / 2 pts

Assuma que você tem uma **Fila Flexível** com as seguintes operações: *void enfileirar(int i)*, *int desenfileirar()*, *boolean vazia()*. Como você pode usar essas operações para simular uma **Pilha Flexível**. Em particular as operações *void empilhar(int i)*, *int desempilhar()*? Apresente as implementações para essas duas operações. Dica: use duas filas, uma principal e outra temporária.

Nesta questão você deve apresentar o código fonte da solução e explicá-lo via vídeo.

Sua Resposta:

```
class Celula {  
    public int elemento; // Elemento inserido na celula.  
    public Celula prox; // Aponta a celula prox.
```

```
public Celula() {  
    this(0);  
}
```

```
public Celula(int elemento) {  
    this.elemento = elemento;  
    this.prox = null;  
}  
}
```

```
class Fila {  
    private Celula primeiro;  
    private Celula ultimo;
```

```
    public Fila() {  
        primeiro = new Celula();  
        ultimo = primeiro;  
    }
```

```
    public void inserir(int x) {  
        ultimo.prox = new Celula(x);  
        ultimo = ultimo.prox;  
    }
```

```
    public int remover() throws Exception {  
        if (primeiro == ultimo) {  
            throw new Exception("Erro ao remover!");  
        }
```

```
        Celula tmp = primeiro;
```

```
primeiro = primeiro.prox;
int resp = primeiro.elemento;
tmp.prox = null;
tmp = null;
return resp;
}

public void mostrar() {
    System.out.print("[ ");
    for(Celula i = primeiro.prox; i != null; i = i.prox) {
        System.out.print(i.elemento + " ");
    }
    System.out.println("] ");
}

public boolean vazia(){
    boolean resp = false;
    if(primeiro == ultimo){
        resp = true;
    }
    return resp;
}

public void enfileirar(int i) {
    Celula tmp = new Celula(i);
    tmp.prox = primeiro;
    primeiro = tmp;
    tmp = null;
}
```

```
public int desenfileirar() throws Exception {
    if (primeiro == null) {
        throw new Exception("Erro ao remover!");
    }
    int resp = primeiro.elemento;
    Celula tmp = primeiro;
    primeiro = primeiro.prox;
    tmp.prox = null;
    tmp = null;
    return resp;
}

public class P02Q09{
    public static void main(String[] args) throws Exception{
        Fila f1 = new Fila();
        Fila tmp = new Fila();

        f1.enfileirar(1);
        f1.enfileirar(2);
        f1.enfileirar(3);
        f1.enfileirar(4);
        f1.enfileirar(5);

        tmp.inserir(1);
        tmp.inserir(2);
        tmp.inserir(3);
        tmp.inserir(4);
        tmp.inserir(5);
    }
}
```

```
System.out.print("f1: ");  
f1.mostrar();  
System.out.print("tmp: ");  
tmp.mostrar();  
  
System.out.println("f1 vazia? " + f1.vazia());  
System.out.println("tmp vazia? " + tmp.vazia());  
  
System.out.println("f1 desenfileirar? " + f1.desenfileirar());  
System.out.println("tmp remover? " + tmp.remover());  
}  
}
```

Nesta questão espera-se que o aluno implemente uma pilha com o auxílio de duas filas para garantir a política **LIFO**. Assim, para **empilhar**, basta que ele chame o enfileirar da fila principal. Para **desempilhar**, o conteúdo da fila principal deve ser movido para temporária até que fique apenas um elemento na fila. Ele então será removido e o conteúdo movido novamente para a fila principal.

Pontuação do teste: **9,5** de 10